Scaling Spherical CNNs: making rotation-equivariant networks big enough to win

Google Research and MIT redesign the core of spherical CNNs and scale them an order of magnitude, reaching state of the art on QM9 and becoming competitive neural weather models.

When we want a network to look at an image, we reach for a convolutional network on a flat grid almost without thinking. But a great deal of scientific data does not live on a plane. A molecule is a cloud of atoms floating in three-dimensional space; the Earth's atmosphere is a field wrapped around a globe. For data like this the right domain is not the plane but the sphere, and the right symmetry is not translation but rotation: a molecule's energy does not change when you spin it, and a weather pattern is the same physical object whether it sits over the Pacific or the Atlantic. Spherical CNNs were built precisely for this setting, replacing planar convolution with a rotation-equivariant convolution on the sphere.

The catch is cost. Spherical convolution is most accurate when it is computed in the spectral domain, as products of Fourier coefficients on the sphere and the rotation group, and that is far more expensive than an ordinary planar convolution. Because of this price tag, spherical CNNs stayed small: low resolution, shallow depth, and modest capacity, tested mostly on tiny benchmarks. There was simply no spherical counterpart to the deep planar networks such as VGG-19 that carry modern computer vision, which left the very symmetry that should have made these models shine locked behind a computational bill that few research groups were willing to pay.

This ICML 2023 paper from Google Research and MIT closes that gap. The authors scale spherical CNNs by a full order of magnitude in both operations and feature resolution, and show that once they are big enough these models can match or beat the equivariant graph neural networks and transformers that dominate molecular and weather modeling. The central lesson is that scaling was not a matter of naively stacking more depth and width: the nonlinearity, the normalization, and the residual block all had to be redesigned, and the whole system rewritten, before a large spherical model could run fast on modern hardware. In what follows we walk through how they did it, and through the evidence that the redesign, rather than the raw increase in size, is what made the difference.

Paper: https://arxiv.org/abs/2306.05420 | Code: https://github.com/google-research/spherical-cnn

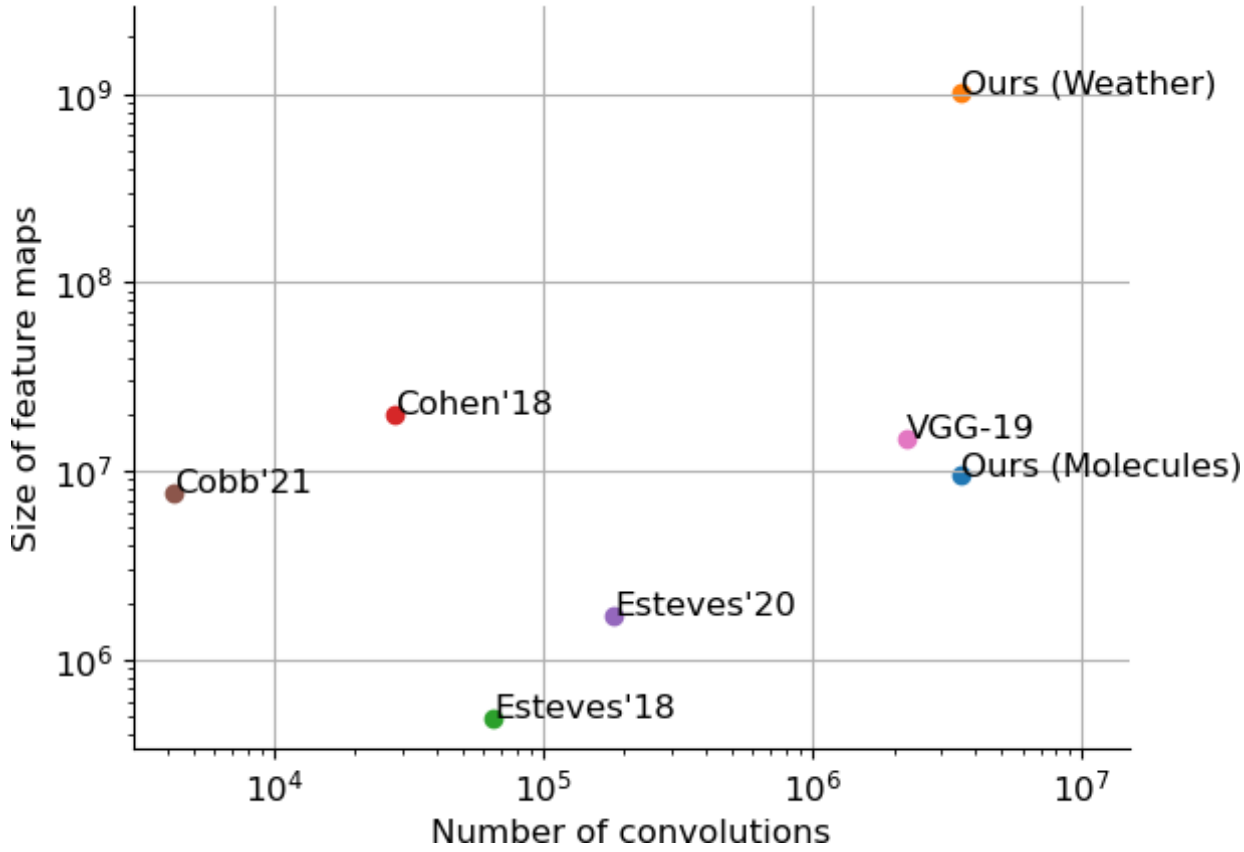

Figure 1. A log-log map of the number of convolutions against feature-map size. Prior spherical CNNs (Esteves'18, Cohen'18, Esteves'20, Cobb'21) sit in the low-compute, low-resolution corner; this work (Ours) reaches an order of magnitude higher, into the range of planar networks such as VGG-19.

Why spherical convolution is expensive

A spherical convolution is defined as an integral over the rotation group SO(3): the filter is rotated by every element of the group and correlated with the signal. Written continuously it reads as an integral of the signal against a rotated kernel over all of SO(3). Discretizing that integral directly is awkward, because there is no arbitrarily dense, perfectly uniform grid on the sphere or on the rotation group. The clean alternative is to move to the spectral domain, where convolution becomes a pointwise product of spherical harmonic coefficients. This is accurate, but the transforms are heavy, and their cost is what had confined spherical CNNs to small problems. Earlier molecular work with spherical CNNs, for instance, used only the QM7 benchmark, which has 7,165 molecules with up to 23 atoms and a single regression target, a small fraction of the scale that graph networks and transformers were already handling with ease.

The recipe for scaling

The paper's contribution is best read as three ingredients that work together: a hardware-tuned implementation, a redesigned set of core layers, and input representations tailored to each application. None of them is a cosmetic tweak; each one improves both speed and

accuracy at the same time, which is exactly the combination a model needs before it can safely grow larger.

An implementation built for the accelerator

The first ingredient is a complete rewrite in JAX of the spin-weighted spherical harmonic transforms, tuned to run fast and distributed on TPUs. The most counterintuitive design decision is to compute the Fourier transforms as dense matrix multiplications rather than fast Fourier transforms. On paper an FFT has lower asymptotic complexity, but on a TPU the matrix-multiply units are extraordinarily fast while shuffling data through memory is the real bottleneck, so a dense matmul wins in practice. The rewrite alone is about 3x faster than the original implementation, and because it runs distributed the authors can spread a model across up to 32 TPUs and speed it up by 100x or more. That headroom is what turns a one-order-of-magnitude jump from a wish into an experiment you can actually run.

Layers that live in the spectral domain

The second ingredient is a set of new general-purpose layers, most of which operate directly on Fourier coefficients. The centerpiece is a phase collapse nonlinearity. Applying an ordinary pointwise nonlinearity on the sphere breaks equivariance; phase collapse instead takes the modulus of the features, which collapses their phase and restores rotation invariance while preserving the information carried in the nonzero spins. Concretely it updates only the spin-zero component through a learned combination of the original feature and its magnitude, and subsequent convolutions propagate that information back to the nonzero spins. The authors also move batch normalization and pooling into the spectral domain, and design an efficient residual block whose skip connection is added directly between Fourier coefficients rather than in the spatial domain.

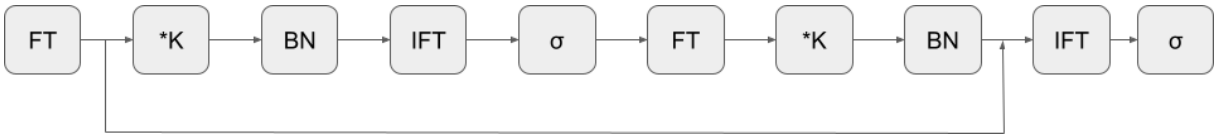

Figure 2. The efficient residual block. Features flow through spin-weighted spherical Fourier transforms (FT) and their inverses (IFT), multiplication with filter coefficients (*K), a phase-collapse activation and spectral batch normalization (BN). The residual connection is added directly in Fourier space rather than in the spatial domain, and optional spectral pooling is folded into the first FT block.

Input representations that fit the data

The third ingredient is application-specific input encodings. For molecules the authors introduce a spherical representation that turns a set of atoms and positions into a set of functions on the sphere. Each atom is described by one spherical channel per atom type, and the value at each point on a sphere comes from physically-based, power-law interactions between pairs of atoms, smoothed with a Gaussian kernel and aggregated over atom types. The result is a rotation-friendly encoding that feeds naturally into the spherical network, rather than a hand-built graph, and it lets the model use the same equivariant machinery that makes it a good fit for the atmosphere.

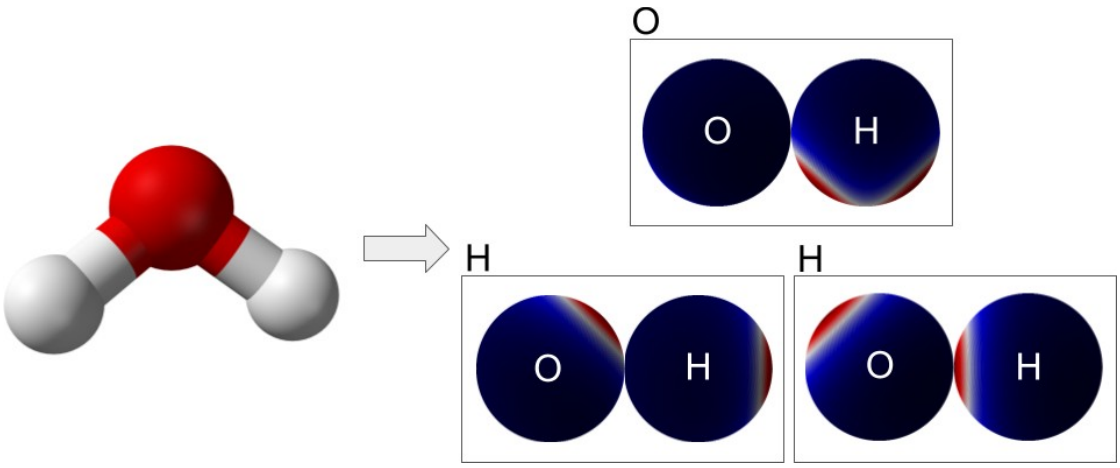

Figure 3. Encoding a molecule as a set of spherical functions. Each atom becomes one spherical channel per atom type (here Z=2 for the H2O example, so two channels per atom). The value at each point on a sphere comes from physically-based interactions between atom pairs, smoothed with a Gaussian kernel and summed over atom types; the sphere marked H, for example, aggregates the Coulomb interactions involving the hydrogen atoms.

Results on molecules and weather

On the molecular side the target is QM9, a benchmark of 134,000 molecules with up to 29 atoms and 12 regression targets spanning energetic, electronic and thermodynamic properties. Scaling spherical CNNs to QM9 for the first time, the model reaches state of the art, outperforming the previously dominant graph neural networks and transformers on 8 of the 12 targets under the first data split and on 9 of the 12 under the second (the two splits differ mainly in training-set size, with the second using 100,000 examples). On the enthalpy of atomization target the model reaches about 15.25 meV mean absolute error.

On the weather side the models are trained on ERA5 reanalysis data through the WeatherBench benchmark, forecasting quantities such as geopotential height (Z500) and temperature (T850, T2M) at 3 and 5 day horizons, with longer tasks reaching out to 28 days. In the simpler two-predictor setting the spherical CNN outperforms the WeatherBench baseline on every metric, and on several temperature metrics it beats even models that were pre-trained on large amounts of simulated data. This is the first demonstration that spherical CNNs are viable neural weather models, though on the hardest iterative forecasts they still trail specialized baselines on some temperature targets.

What actually mattered: an ablation

Because the authors argue that redesign, not brute-force scaling, is the key, the ablation is the most instructive table in the paper. Starting from the JAX implementation as a baseline, each new component is switched on in turn, and the table reports its effect on both throughput (steps per second) and error (RMSE) on a QM9 model. The phase collapse activation alone cuts error by 8.0%, spectral batch normalization trims a further 1.4%, and the efficient residual block another 2.4%, and none of them costs accuracy for speed: the residual block and normalization actually run faster while helping the error.

Table 1. Effect of each modeling contribution on a QM9 model, measured as the change relative to the previous row. Positive step/s is faster; negative RMSE is more accurate.

Contribution	Delta Steps/s [%]	Delta RMSE [%]
JAX implementation (baseline)	33.7	0.0
+ Phase collapse	-4.6	-8.0
+ Spectral batch norm	7.8	-1.4
+ Efficient residual block	19.3	-2.4

Steps/s for the JAX row is the absolute throughput; later rows show the change from adding each component. A separate comparison confirms that phase collapse, spectral pooling and the new molecule representation each beat their prior alternatives, together driving the QM9 enthalpy error to 15.25 meV.

Takeaway

The clean summary of this work is that spherical CNNs were never fundamentally limited; they were simply poorly scaled. With a redesigned nonlinearity, normalization and residual block, and an implementation tuned to the way accelerators actually spend their time, these models finally reach the scale of real scientific problems, setting the state of the art on molecular property prediction and becoming genuinely competitive neural weather models. There is still a cost: the spectral convolutions remain heavy, training a large model can take days on 32 TPUs, and weather forecasting is not yet solved. But the authors release their JAX implementation as a platform, and the more interesting message is a direction, not a single result: for data that is naturally spherical and rotation-related, equivariance is worth scaling up rather than approximating away.